PRACTICAL 8

Title :
Structured Testing - Loop Coverage, Call Coverage and Path Coverage

Problem Statement:
For foll program, draw path coverage diagram, determine cyclomatic complexity write basic paths to be tested and test cases

```
Euclid(int m, int n)
    int r;
      if(n>m) {
          r = m;
          m = n;
          n = r;
    }
    r = m % n;
    while (r != 0) {
          m = n;
          n = r;
          r = m % n;
      }
    return n;
  }
```

Theory:

Control Flow graph - Describes logic structure of software modules. A module corresponds to a single function or subroutine in typical languages has a single entry and exit point and is able to be used as a design component through call / return mechanism

Def of Cyclomatic Complexity-
It is a software metric that measures the complexity of a programs control flow. It is calculated by counting the no. of linearly independent paths through a programs source code.

Cyclomatic Complexity:

$V(G) = E - N + 2$
$= 15 - 14 + 2$
$= 3$

Basis Test Path:

3 Paths

Test Path B1: 0 1 5 6 7 11 12 13

8(1): n > m → False
14(7): r! = 0 → False

Test Path B2: 0 1 2 3 4 5 6 7 11 12 13

8(1): n > m → True
14(7): r! = 0 → False

Test Path B3: 0 1 5 6 7 8 9 10 7 11 12 13

8(1): n > m → False
14(7): r! = 0 → True
14(7): r! = 0 → False

Conclusion:

The control flow graph is drawn and cyclomatic complexity is determined to be 3. Three basis paths are to be tested

PRACTICAL 9

Title:
Data Flow Testing

Problem Statement:

```
void quad_eqn (float A,B,C, Boolean Is_Complex)
{float Discrim = B*B-4*A*C
float R1,R2;
{
If Discrim<0.0
Is_Complex = true;
Else
Is_Complex = false;
Endif;
If not Is_Complex
                R1=(-B+Sqrt(Discrim))/(2.0*A)
                ;
                R2=(-B-Sqrt(Discrim))/(2.0*A)
                ;
Endif;
End quad_eqn;}
}
```

For the above function, draw foll tables and write test cases:
a. Occurrence of variables and their categories
b. du pairs and their type
c. All c-uses
d. All p-uses

Theory:
Data flow testing is a structural testing technique that
- Aims to execute sub-paths from points where each variable is defined to points where it is referenced
- These sub-paths are called definition-use pairs or du-pairs

Most failures involve execution of an incorrect definition
- Incorrect assignment or input statement
- Incorrect path is taken, which leads to incorrect definition
- Definition is missing

Terminology:

• Variable definition
Occurrences of a variable where a variable is given a new value.

• Variable uses
Occurrences of a variable where a variable is not given a new value

• p-uses (predicate uses)
Occur in predicate portion of a decision statement such as if-then-else, while-do etc

• c-uses (computation uses)
All others, including variable occurrences in the right hand side of an assignment statement or an output statement

• du-path
A sub-path from a variable definition to its use

Data Flow testing: Key steps
Given a code (program or pseudo-code)
1. Number the lines
2. List the variables
3. List occurrences and assign a category to each variable
4. Identify du-pairs and their use (p- or c-)
5. Define test cases, depending on required coverage

No. : 26

Date :

Solution :

Step 1 - Number the lines

```
0  void quad_eqn (float A,B,C, Boolean Is_Complex)
1  {float Discrim = B*B-4*A*C
2  float R1,R2;
3  {
4  If Discrim <0.0
5  Is_Complex = true;
6  Else
7  Is_Complex = false
8  Endif;
9  If not Is_Complex
                R1 = (-B+Sqrt(Discrim))/(2.0*A)
10              ;
                R2 = (-B-Sqrt(Discrim))/(2.0*A)
11              ;
12 Endif;
13 End quad_eqn; }
14 }
```

Step 2 - List variables

- A, B, C
- DISCRIM
- Is_Complex
- R1, R2

Conclusion :

13 test cases were designed for data flow testing

PRACTICAL 10

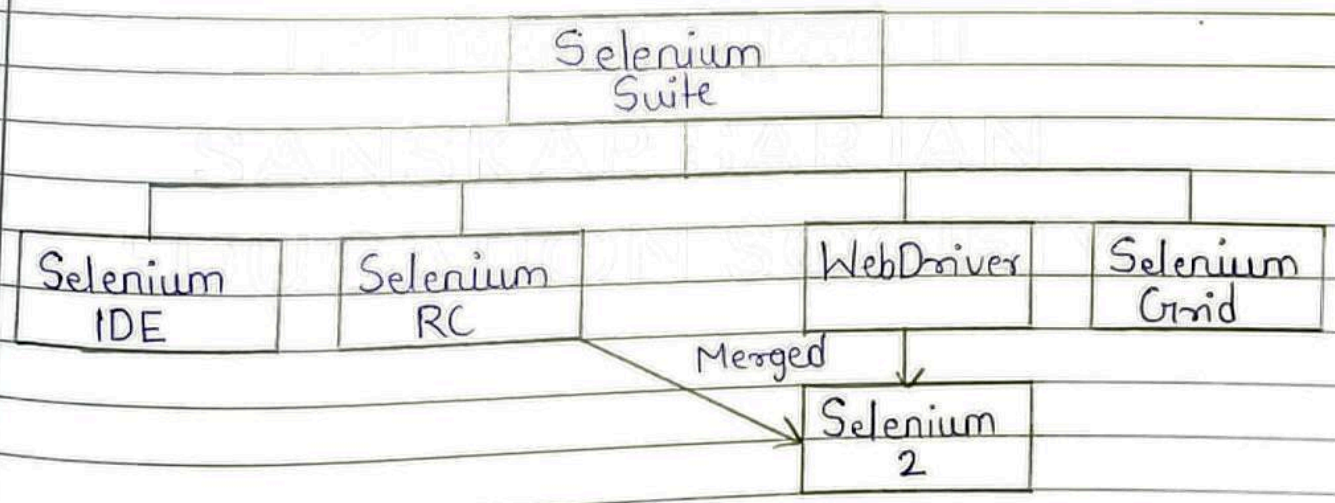
Test Automation using Selenium IDE

Problem Statement:
Automate web test using Selenium IDE and Firebug

Theory:
Selenium is a free open source automated testing suite for web applications across different browsers and platforms
Selenium is a suite of software, each catering to different testing needs of an organization. It has four components.

- Selenium Integrated Development Environment (IDE)
- Selenium Remote Control (RC)
- WebDriver
- Selenium Grid

Selenium IDE is the simplest framework in the Selenium suit and is the easiest one to learn. It is a Firefox plugin that you can install as easily as you can with other plugins.
Selenium IDE should only be used as a prototyping tool



Test Case 1 : Positive Login test

① Open Page

② Type username 'student' into Username field

③ Type password 'Password123' into Password field

④ Push Submit button

⑤ Verify new page URL contains practicetestautomation.com/logged-in-successfully/

⑥ Verify new page contains expected text

⑦ Verify button Log out is displayed on the new page

Test Case 2 : Negative Username test

① Open Page

② Type username 'students' into Username field

③ Type password 'Password123' into Password field

④ Push Submit button

⑤ Verify error message is displayed

⑥ Verify error message text is Your username is invalid!

Test Case 3: Negative Password test

① Open Page

② Type username 'student' into Username field

③ Type password 'Password1' into Password. field

④ Push Submit button

⑤ Verify error message is displayed

⑥ Verify error message text is Your password is invalid!

Test Case 4: Negative username and password test

① Open Page

② Type username 'students' into Username field

③ Type password 'Password1' into Password field

④ Push Submit button

⑤ Verify error message is displayed

⑥ Verify error message text is Your username & password is invalid!